

Manifold as Deterministic Runtime: How Mathematical Structure Implies Execution Semantics

*A Formal Correspondence Between Differential Geometry and
Computational Execution Models*

Leon Calvin Long, II

Technical Paper · September 2026 · Version 1.1

Drafted with AI assistance (Microsoft Copilot); audited, corrected, and revised by the author.

Abstract

We advance the thesis that a smooth manifold $(M, \mathcal{A})$, far from being merely a geometric backdrop, constitutes a fully deterministic runtime environment by virtue of its intrinsic mathematical structure. The smooth atlas, chart-transition maps, tangent bundle, and fiber bundle formalism together impose a precise execution semantics on any computational process that inhabits or traverses the manifold. We formalize a correspondence between four canonical geometric properties and four computational invariants: (i) coordinate invariance of geometric quantities corresponds to referential transparency and pure functional computation; (ii) the chart-overlap gluing condition corresponds to the consistency requirement of deterministic finite-state automata; (iii) geodesic minimization under a torsion-free connection corresponds to optimal, least-cost execution paths in a directed computation graph; and (iv) curvature constraints bound the degree to which nondeterminism can arise within any locally flat region. We argue that runtime singularities — points at which the manifold fails to be smooth — are the geometric analogue of undefined behavior or nondeterministic branching. Consequently, determinism is not a runtime policy that must be enforced by the

programmer or scheduler; it is a topological invariant of the underlying space. We conclude with an implementation sketch and a survey of related foundational work in directed topology, cohesive homotopy type theory, and categorical semantics.

Keywords: *smooth manifold, runtime semantics, determinism, fiber bundle, geodesic, referential transparency, differential programming, topological invariant, type theory*

1. Introduction

The question of whether geometric structure can constrain computational behavior is not merely philosophical. Runtime systems are defined by their state spaces, their transition rules, and the properties — determinism, termination, type safety — that hold across all executions. The central observation of this paper is that a smooth manifold already encodes all of these components as theorems, not design decisions.

Differential geometry has long served as the mathematical substrate for physical dynamics: a point particle traces a geodesic; a field configuration is a section of a fiber bundle; symmetry groups act smoothly on state spaces. What has been less systematically exploited is the precision with which these structures map onto the semantics of computation. A chart (U, φ) in an atlas is not merely a local coordinate patch — it is a scoped execution context with well-defined entry and exit conditions. A smooth transition map $\varphi_\beta \circ \varphi_\alpha^{-1}$ is not merely a change of coordinates — it is a deterministic, invertible state transition. The tangent bundle TM is not merely the space of velocity vectors — it is the first-order derivative of computation, the space in which program sensitivities live.

The purpose of this paper is to make these correspondences precise, to derive formal consequences from them, and to argue that a manifold, properly instantiated, *is* a runtime — one whose determinism is a theorem of differential topology.

Contributions. This paper makes the following specific contributions:

1. A formal dictionary between smooth manifold structure and execution semantics (Section 3), covering charts, transition maps, tangent vectors, geodesics, and fiber bundles.
2. A proof sketch that determinism is a topological invariant: orientable, simply connected manifolds admitting a global chart are provably deterministic runtimes (Section 4).
3. The introduction of the concept of a *runtime singularity* as the geometric analogue of undefined behavior and nondeterministic branching (Section 4).
4. A constructive implementation sketch outlining how the manifold-runtime correspondence can be realized in a practical system (Section 5).
5. A structured survey of related foundational work in directed topology, cohesive homotopy type theory, and categorical abstract machines (Section 6).

2. Mathematical Preliminaries

We establish notation and recall the core definitions from differential geometry that underpin the subsequent correspondence. Throughout, all manifolds are assumed Hausdorff, second-countable, and smooth (class C^∞) unless otherwise specified. We follow the conventions of Lee [L03, GP74] and Milnor–Stasheff [MS74].

2.1 Smooth Manifolds and Atlases

Definition 2.1 (Smooth Manifold).

A

smooth manifold of dimension n

is a pair

$$(M, \mathcal{A})$$

where

$$M$$

is a topological space and

$$\mathcal{A} = \{(U_\alpha, \varphi_\alpha)\}_{\alpha \in I}$$

is a maximal smooth atlas: a collection of charts such that (i)

$$\{U_\alpha\}$$

covers

$$M$$

; (ii) each

$$\varphi_\alpha : U_\alpha \rightarrow \mathbb{R}^n$$

is a homeomorphism onto an open set; and (iii) for all

$$\alpha, \beta$$

with

$$U_\alpha \cap U_\beta \neq \emptyset$$

, the transition map

$$\varphi_\beta \circ \varphi_\alpha^{-1} : \varphi_\alpha(U_\alpha \cap U_\beta) \rightarrow \varphi_\beta(U_\alpha \cap U_\beta)$$

is a

$$C^\infty$$

diffeomorphism.

Definition 2.2 (C^∞ Compatibility).

Two charts

$$(U_\alpha, \varphi_\alpha)$$

and

$$(U_\beta, \varphi_\beta)$$

are

smoothly compatible

if their overlap transition map is

$$C^\infty$$

. A maximal collection of mutually compatible charts is a

smooth structure

on

$$M$$

.

2.2 Tangent Structure

Definition 2.3 (Tangent Space).

For

$$p \in M$$

, the

tangent space

T_pM

is the vector space of derivations on the algebra

$C^\infty(M)$

at

p

: smooth linear maps

$v : C^\infty(M) \rightarrow \mathbb{R}$

satisfying the Leibniz rule

$v(fg) = f(p)v(g) + g(p)v(f)$

.

Definition 2.4 (Tangent Bundle).

The

tangent bundle

is the disjoint union

$TM = \sqcup_{p \in M} T_pM$

, equipped with a natural smooth manifold structure of dimension

$2n$

, and the canonical projection

$\pi : TM \rightarrow M$

,

$\pi(p, v) = p$

. The

cotangent bundle

$$T^*M$$

is defined dually, with fibers

$$T_p^*M = (T_pM)^*$$

.

2.3 Fiber Bundles and Connections

Definition 2.5 (Fiber Bundle).

A

fiber bundle

is a tuple

$$(E, B, \pi, F)$$

where

$$E$$

(total space),

$$B$$

(base space), and

$$F$$

(typical fiber) are smooth manifolds, and

$$\pi : E \rightarrow B$$

is a smooth surjection that is locally trivial: for each

$$b \in B$$

there exists an open neighborhood

$$V \ni b$$

and a diffeomorphism

$$\Phi : \pi^{-1}(V) \rightarrow V \times F$$

with

$$pr_1 \circ \Phi = \pi$$

. A

section

is a smooth map

$$\sigma : B \rightarrow E$$

with

$$\pi \circ \sigma = id_B$$

.

Definition 2.6 (Connection and Parallel Transport).

A

connection

$$\nabla$$

on a vector bundle

$$E \rightarrow M$$

is a bilinear map

$$\nabla : \Gamma(TM) \times \Gamma(E) \rightarrow \Gamma(E)$$

satisfying (i)

$$C^\infty(M)$$

-linearity in the first argument and (ii) the Leibniz rule

$$\nabla_X(f\sigma) = (Xf)\sigma + f\nabla_X\sigma$$

for all

$$f \in C^\infty(M)$$

.

Parallel transport

of a section along a curve

$$\gamma$$

is the solution to

$$\nabla_{\dot{\gamma}}\sigma = 0$$

.

Definition 2.7 (Geodesic).

A smooth curve

$$\gamma : I \rightarrow M$$

on a Riemannian manifold

$$(M, g)$$

with Levi-Civita connection

$$\nabla$$

is a

geodesic

if it satisfies the autoparallel condition:

$$\nabla_{\dot{\gamma}} \dot{\gamma} = 0$$

. Equivalently, in local coordinates

$$x^k$$

, this reads

$$\dot{\gamma}^k + \Gamma^k_{ij} \dot{\gamma}^i \dot{\gamma}^j = 0$$

, where

$$\Gamma^k_{ij}$$

are the Christoffel symbols of

$$\nabla$$

.

2.4 Notation Reference Table

Symbol	Meaning	Computational Analogue
$(M, \mathcal{A})$	Smooth manifold with atlas	Runtime environment with execution contexts
$(U_\alpha, \varphi_\alpha)$	Chart: open set with coordinate map	Scoped execution context / stack frame
$\varphi_\beta \circ \varphi_\alpha^{-1}$	Transition map between charts	Deterministic state transition function
$T_p M$	Tangent space at p	Space of program derivatives at a state
TM	Tangent bundle	Full space of differentiable program sensitivities
T^*M	Cotangent bundle	Space of observable

Symbol	Meaning	Computational Analogue
		outputs / dual program states
(E, B, π, F)	Fiber bundle	Typed program over an input space
$\sigma : B \rightarrow E$	Section of a fiber bundle	Program: consistent typed assignment over inputs
∇	Connection on a vector bundle	Type-transformation rule between execution contexts
$\nabla_{\dot{\gamma}} \dot{\gamma} = 0$	Geodesic equation	Optimal (shortest-path) execution trace
$R(X, Y)Z$	Riemann curvature tensor	Measure of nondeterminism / path non-commutativity
$T(X, Y) = \nabla_X Y - \nabla_Y X - [X, Y]$	Torsion tensor	Asymmetry of parallel composition
$H^k_{dR}(M)$	de Rham cohomology	Global runtime consistency invariants

3. The Runtime Correspondence

We now develop the central technical content of the paper: a systematic formal correspondence between the structure of a smooth manifold and the semantics of a deterministic runtime. Each subsection establishes one component of the dictionary.

3.1 Charts as Execution Contexts

A chart $(U_\alpha, \varphi_\alpha)$ assigns to each point $p \in U_\alpha$ a coordinate representation $\varphi_\alpha(p) \in \mathbb{R}^n$. Within this chart, all smooth functions on M admit a local expression in terms of the coordinate variables $(x^1, \dots, x^n) = \varphi_\alpha(p)$. This structure is precisely that of a *scoped execution context*: a bounded environment in which variable names are resolved to values, and all expressions reduce to their local coordinate form.

The critical property is that the choice of chart does not affect the intrinsic value of any geometric quantity. A scalar function $f \in C^\infty(M)$ has a single well-defined real value at each point, regardless of which chart is used to compute it. This is the analogue of *referential transparency* in the functional programming sense: the value of an expression depends only on its inputs, not on the execution environment that evaluates it.

The boundary of a chart — the frontier of U_α within M — corresponds to the boundary of a scope. When execution reaches this boundary, a transition map must be invoked to pass control to an adjacent chart. The smoothness of this transition guarantees *semantic continuity*: no information is lost or corrupted at context boundaries.

3.2 Determinism from Coordinate Invariance

A fundamental principle of differential geometry is that geometric objects — tensors, forms, connections — are coordinate-invariant. Their coordinate representations change under chart transitions, but the underlying objects do not. This invariance is the precise mathematical content of *determinism* in the computational sense.

Proposition 3.1 (Coordinate Invariance → Referential Transparency).

Let

$$f : M \rightarrow \mathbb{R}$$

be a smooth function. For any two charts

$$(U_\alpha, \varphi_\alpha)$$

and

$$(U_\beta, \varphi_\beta)$$

with

$$p \in U_\alpha \cap U_\beta$$

, the representations

$$f \circ \varphi_\alpha^{-1}$$

and

$$f \circ \varphi_\beta^{-1}$$

satisfy:

$$(f \circ \varphi_\alpha^{-1})(\varphi_\alpha(p)) = (f \circ \varphi_\beta^{-1})(\varphi_\beta(p)) = f(p).$$

Proof sketch.

By definition,

$$\varphi_\beta \circ \varphi_\alpha^{-1}$$

is a

$$C^\infty$$

diffeomorphism on the overlap. Composing with

$$f \circ \varphi_\beta^{-1}$$

yields

$$f \circ \varphi_\beta^{-1} \circ (\varphi_\beta \circ \varphi_\alpha^{-1}) = f \circ \varphi_\alpha^{-1}$$

on the overlap. Since both sides evaluate to the same real number

$$f(p)$$

, the value is chart-independent. ■

Computationally, Proposition 3.1 states that any expression that corresponds to a smooth function on the manifold will return the same value regardless of the execution context (chart) in which it is evaluated — provided the execution context is valid (i.e., the point lies in the chart's domain). This is exactly referential transparency: equal inputs produce equal outputs, with no dependence on ambient state.

3.3 Transition Maps as State Transitions

The gluing condition on chart overlaps — that transition maps must be C^∞ diffeomorphisms — encodes a strong consistency requirement. Specifically, consider two overlapping charts $(U_\alpha, \varphi_\alpha)$ and (U_β, φ_β) . On the overlap $U_\alpha \cap U_\beta$, the two coordinate representations must be related by a bijective, smooth, smoothly invertible map. No point in the overlap is ambiguous, and no information is destroyed.

This is formally identical to the transition function of a *deterministic finite automaton* (DFA) or, more generally, a *deterministic state machine*: from any state in the overlap, there is exactly one successor state in each adjacent chart, determined by the transition map. The smoothness requirement strengthens this: not only is the transition deterministic, it is also *continuous and differentiable*, meaning small perturbations in the pre-transition state produce bounded, predictable perturbations in the post-transition state.

Proposition 3.2 (Gluing Condition $\rightarrow$ State Machine Consistency).

Let

$$\tau_{\alpha\beta} = \varphi_\beta \circ \varphi_\alpha^{-1}$$

be a transition map. Then

$$\tau_{\alpha\beta}$$

is: (i) total on its domain

$$\varphi_\alpha(U_\alpha \cap U_\beta)$$

; (ii) injective (reversible: distinct inputs have distinct outputs);

(iii) surjective onto

$$\varphi_\beta(U_\alpha \cap U_\beta)$$

; and (iv) invertible via

$$\tau_{\beta\alpha} = \tau_{\alpha\beta}^{-1}$$

. These are precisely the properties of a deterministic, reversible state transition. ■

Reversibility is a non-trivial bonus: the manifold structure not only guarantees deterministic forward transitions, it guarantees deterministic backward transitions as well. This corresponds to time-reversible computation in the sense of Landauer and Bennett, and is closely related to the absence of information loss — i.e., garbage-free computation [Lan96].

3.4 Tangent Vectors as Computational Derivatives

By Definition 2.3, a tangent vector $v \in T_p M$ is a derivation: a linear map on smooth functions at p satisfying the Leibniz rule. In the coordinate basis $\{\partial/\partial x^i\}$ induced by a chart, $v = v^i \partial/\partial x^i$ acts on a function f by $v(f) = v^i \partial f / \partial x^i$. This is the directional derivative of f in the direction v .

The computational analogue is precise and well-known in the context of *differentiable programming* and *automatic differentiation* (AD). In forward-mode AD, a program is augmented with a dual component — its directional derivative with respect to an input perturbation. The tangent vector at a program state is exactly this dual component: it records the *rate of change of the computation* with respect to variations in input. The Leibniz rule for derivations corresponds to the product rule for derivatives, which is the fundamental correctness condition of forward-mode AD.

The cotangent bundle T^*M correspondingly captures reverse-mode AD (backpropagation): a cotangent vector (one-form) $\omega \in T_p^* M$ acts on tangent vectors to produce scalars, which is precisely how gradients operate on perturbations in the adjoint method.

The tangent bundle

TM

is, in this sense, the canonical space of program derivatives. A smooth vector field

$$X \in \Gamma(TM)$$

— a smooth assignment of a tangent vector to each point — is a differentiable program: it specifies, at every state, the direction and magnitude of computation.

3.5 Geodesics as Optimal Execution Paths

Given a Riemannian metric g on M , the length of a curve $\gamma: [0,1] \rightarrow M$ is $L(\gamma) = \int_0^1 \sqrt{g_{\gamma(t)}(\dot{\gamma}(t), \dot{\gamma}(t))} dt$. A geodesic is a critical point of this length functional — equivalently, a curve of locally minimal energy $E(\gamma) = \int_0^1 g_{\gamma(t)}(\dot{\gamma}(t), \dot{\gamma}(t)) dt$. The geodesic equation $\nabla_{\dot{\gamma}} \dot{\gamma} = 0$ characterizes these curves intrinsically.

Computationally, an execution trace is a path through the state space. Given a cost metric on the state space (e.g., time complexity, memory usage, communication overhead), the optimal execution trace is the one that minimizes total cost. In a Riemannian manifold model of a computation space, this optimal trace is precisely the geodesic between the initial and terminal states.

Proposition 3.3 (Torsion-Free Connection → Deterministic Optimal Paths).

If the connection

$$\nabla$$

on

$$(M, g)$$

is the Levi-Civita connection (metric-compatible and torsion-free), then: (i)

geodesics are locally unique given initial position and velocity; (ii) the exponential map

$$\exp_p : T_p M \rightarrow M$$

is a local diffeomorphism in a neighborhood of

0

; and (iii) parallel transport along geodesics is isometric. Consequently, the optimal execution path from any state

p

in direction

v

is uniquely determined by

(p, v)

— there is no nondeterministic choice of path. ■

The torsion-free condition $T(X, Y) = \nabla_X Y - \nabla_Y X - [X, Y] = 0$ is particularly significant: it means that the order in which two infinitesimal operations are composed does not affect the outcome to first order. In concurrent computation, this is the definition of *commutativity of concurrent operations* — the diamond property of the Church-Rosser theorem, or confluence in rewriting theory. A torsion-free runtime is, in this precise sense, a confluent runtime.

3.6 Fiber Bundles as Typed Computations

The fiber bundle (E, B, π, F) provides the natural geometric model for a typed program. The base space B is the space of inputs or program states. The fiber $F_b = \pi^{-1}(b)$ over each base point $b \in B$ is the *type* at that state: the set of valid computational values that can exist at b . A section $\sigma: B \rightarrow E$ — a smooth, consistent assignment of a point in F_b to each $b \in B$ — is a *program*: it assigns a well-typed output value to every input.

The connection ∇ on the bundle defines how types transform as the base point moves. Parallel transport along a path γ in B lifts the path to the total space E , transporting a fiber element smoothly. This is the geometric model of *type coercion along a computation path*: as the program moves through state space, the type of its current

value transforms according to the connection, in a way that is smooth and invertible (since parallel transport is an isomorphism on fibers) [KN63].

Vector bundles (where $F = \mathbb{R}^k$) model programs with linearly-structured types — analogous to polymorphic functions over linear types. Principal bundles (where $F = G$, a Lie group) model programs whose type structure has a group symmetry — analogous to parametric polymorphism or group-equivariant computation.

4. Determinism as a Topological Invariant

Having established the component-wise correspondence between manifold structure and execution semantics, we now argue the central thesis: that determinism is not a behavioral property to be enforced at runtime, but a *topological invariant* of the underlying space.

4.1 Conditions for Global Determinism

Theorem 4.1 (Topological Determinism).

Let

(M, g)

be a Riemannian manifold that is: (i)

orientable

(the tangent bundle admits a nowhere-zero top-degree form); (ii)

simply connected

(

$\pi_1(M) = 0$

); and (iii) admits a

global chart

(i.e., is diffeomorphic to an open subset of

$\mathbb{R}^n$

). Then any complete smooth dynamical system on M

M

is globally deterministic: for any initial condition

$(p_0, v_0) \in TM$

, the integral curve of a smooth vector field through

p_0

in direction

v_0

is unique and globally defined.

Proof sketch.

Simple connectivity eliminates topological obstructions to global uniqueness (no nontrivial loops around which parallel transport could produce holonomy).

Orientability ensures consistent global orientation of computation (no unresolvable branching due to orientation reversal). The global chart condition eliminates chart-transition nondeterminism entirely: there is a single, globally valid coordinate system. Under these conditions, the Picard-Lindelöf theorem applies locally: the ODE system has a unique solution for all time, extended globally by completeness.

4.2 Runtime Singularities

A *runtime singularity* is defined as a point $p \in M$ at which the manifold structure fails — specifically, a point at which no chart is defined (a singularity in the topological

sense), or at which the metric tensor degenerates ($\det(g_{ij}) \rightarrow 0$ or $\rightarrow \infty$). At such a point, the geodesic equation becomes ill-defined, the exponential map fails to be a local diffeomorphism, and the uniqueness guaranteed by Theorem 4.1 collapses.

Computationally, this corresponds precisely to: **(a) undefined behavior** — execution reaches a state for which no valid transition is defined; **(b) nondeterministic branching** — two or more equally valid continuations exist and no canonical choice is determined by the structure; or **(c) type errors** — the fiber over the current base point is empty or ill-formed, so no valid section value exists.

A well-designed manifold runtime therefore requires that the underlying manifold be *geodesically complete* (every geodesic extends to all of $\mathbb{R}$, by the Hopf-Rinow theorem) and *non-singular*. These are design constraints on the state space — not on the runtime scheduler.

4.3 de Rham Cohomology as Global Runtime Verification

The *de Rham cohomology* groups $H^k_{dR}(M)$ measure global topological obstructions on M : a closed k -form that is not exact witnesses the existence of a nontrivial k -cycle — a loop that cannot be contracted. In runtime terms, a nontrivial element of $H^1_{dR}(M)$ corresponds to a nontrivial execution loop around which parallel transport fails to return to its starting point — i.e., a computation that produces different results depending on which path was taken through the state space.

By *Stokes' theorem*, $\int_{\partial\Omega} \omega = \int_{\Omega} d\omega$ for any $(k+1)$ -chain Ω and k -form ω . When $d\omega = 0$ (the form is closed) and $H^k_{dR}(M) = 0$ (the cohomology is trivial), every closed form is exact — every local conservation law extends globally. This is the geometric statement of *global runtime consistency*: any local invariant of the computation extends to a global invariant, without contradiction.

Consequently, computing the de Rham cohomology of the state space manifold is — in principle — a method for *static verification* of global runtime consistency properties.

5. Implementation Sketch

We describe, at an architectural level, how the manifold-runtime correspondence could be realized in a concrete computational system. This sketch is not a full engineering specification; it is intended to demonstrate the constructive content of the theory.

5.1 State Space as a Riemannian Manifold

The runtime state space is modeled as a Riemannian manifold (M, g) . Points in M represent program states. The metric g encodes a cost structure on the state space: the length of a path measures execution cost (time, memory, communication). Candidate implementations for M include: finite-dimensional smooth manifolds for programs with bounded state (e.g., fixed-width numerical computation), and infinite-dimensional Fréchet manifolds or Hilbert manifolds for higher-order or functional programs.

5.2 Operations as Smooth Vector Fields

Each primitive computational operation is realized as a smooth vector field $X \in \Gamma(TM)$. Composition of operations corresponds to the Lie bracket or flow composition of vector fields. The condition that operations commute corresponds to $[X, Y] = 0$ — a flat Lie algebra — which is verifiable statically from the algebraic structure of the operations.

5.3 Execution Trace as an Integral Curve

A program execution is an integral curve of a vector field: a curve $\gamma: \mathbb{R} \rightarrow M$ satisfying $\dot{\gamma}(t) = X_{\gamma(t)}$ for all t . By the Picard-Lindelöf theorem, this curve is unique given an initial condition — the runtime's execution is deterministic. The program terminates when $\gamma(t_*)$ reaches a fixed point of X (i.e., $X_p = 0$).

Recent control-theoretic work treats system trajectories as manifold points [Bay26].

5.4 Type System as a Vector Bundle

The type system is modeled as a vector bundle $E \rightarrow M$. The fiber E_p over each state p is the type of the program's current value at state p . A well-typed program is a smooth section $\sigma \in \Gamma(E)$. Type-checking corresponds to verifying that σ is a well-defined

smooth section. Type inference corresponds to constructing a section from local data and verifying its global smoothness.

5.5 Compiler as a Chart-Generating Map

The compiler maps source code to a chart (U, φ) : it assigns to a program fragment a local coordinate representation on the manifold. Linking and module composition correspond to gluing charts along transition maps. The compilation of a complete program is the construction of a maximal atlas covering the relevant region of M , with all transition maps verified to be smooth diffeomorphisms.

5.6 Analogues in Existing Systems

Manifold-Runtime Concept	Analogous Existing System/Framework
Smooth functions as pure programs	Haskell: purely functional programs, referential transparency
Tangent/cotangent bundle, AD	JAX (forward/reverse mode AD), PyTorch autograd
Fiber bundle typed computation	Dependent type theory (Agda, Coq), linear type systems
Categorical semantics, section as program	Topos theory (Lawvere, Tierney), categorical abstract machines
Directed paths in state space	Directed topology (Grandis), process algebras with partial orders
Cohesive types, smooth types	Cohesive HoTT (Schreiber, Shulman), synthetic differential geometry (Kock)

6. Related Work

6.1 Synthetic Differential Geometry and Smooth Types

Lawvere's categorical dynamics [Law79] introduced the idea that dynamical systems could be studied in a categorical setting, where smooth maps and their composition form the primary structure. Kock's synthetic differential geometry [Koc81] replaced

classical set-theoretic foundations with a topos-theoretic setting in which the real number line contains nilpotent infinitesimals, making the Leibniz rule an axiom rather than a limit theorem. This provides a foundation for the derivation interpretation of tangent vectors (Definition 2.3) that is formally cleaner than the classical approach. Escardó's work on the universal property of the interval and related spaces extends these ideas toward the semantics of programming languages.

6.2 Cohesive Homotopy Type Theory

Schreiber [Sch13] and Shulman [Shu18] developed cohesive homotopy type theory (cohesive HoTT), a modal extension of HoTT in which spatial and smooth structure is encoded by a quartet of adjoint modalities (shape $\mathcal{S}$, flat $\flat$, sharp $\sharp$, and infinitesimal shape $\mathfrak{J}$). Within this framework, smooth manifolds, fiber bundles, connections, and differential forms all have direct type-theoretic representations. The cohesive HoTT framework provides the closest existing foundational system to the manifold runtime proposed here: types are smooth spaces, programs are smooth maps, and type-checking is the verification of smoothness and section conditions.

6.3 Directed Topology and Geometric Semantics for Concurrency

Fajstrup, Goubault, and Raussen [FGR16] developed a geometric semantics for concurrent programs based on directed topological spaces (*d-spaces*), where execution traces are directed paths and the space of execution traces has the topology of a path space. Their work demonstrates that properties of concurrent programs — deadlock-freedom, reachability, progress — correspond to topological properties of the directed state space: non-empty path spaces, connected components, and homotopy classes of directed paths. This is a direct precursor to our framework; the main generalization here is the introduction of a smooth (differentiable) structure beyond the topological setting, enabling the use of differential-geometric tools (geodesics, curvature, connections) rather than purely topological ones.

6.4 Categorical Abstract Machines

Curien's categorical abstract machine (CAM) [Cur86] provides a denotational semantics for the lambda calculus in which programs are morphisms in a Cartesian

closed category. The compilation of a lambda term to a CAM instruction sequence is a syntactic translation of type derivations into categorical composition. While the CAM operates in a purely algebraic setting (without differential structure), it establishes the foundational principle that the categorical structure of the semantic domain — not the syntax of the language — determines the execution semantics. Our manifold runtime extends this principle to the smooth category: the smooth manifold structure of the state space determines the execution semantics, and the compiler's role is to embed programs faithfully into this structure [Mac98].

6.5 Characteristic Classes and Global Invariants

Milnor and Stasheff [MS74] developed the theory of characteristic classes — topological invariants of vector bundles that obstruct certain global structures. The Euler class obstructs the existence of a nowhere-zero section; the Chern classes (for complex bundles) measure the twisting of the bundle. In runtime terms, a nontrivial Euler class obstructs the existence of a globally well-typed program (a nowhere-zero section of the type bundle); nontrivial Chern classes indicate that the type structure is globally twisted and cannot be trivialized by any choice of basis. Characteristic class computations thus provide another avenue for static verification of global program properties.

7. Conclusion

We have argued, with formal precision, that a smooth manifold $(M, \mathcal{A})$ equipped with a Riemannian metric and compatible connection constitutes a fully deterministic runtime environment. This determinism is not imposed by a scheduler, enforced by a type-checker, or guaranteed by a programming discipline: it is a theorem of the underlying geometry. Charts are execution contexts; transition maps are state transitions; tangent vectors are program derivatives; geodesics are optimal execution paths; fiber bundles are typed programs; and de Rham cohomology is the invariant theory of global runtime consistency.

The central claim can be stated definitively: *a manifold, properly instantiated as a computational substrate, is not merely a model of a deterministic runtime — it is the runtime, and its determinism is a consequence of its topology and differential structure, not of any engineering decision.*

Future Directions. Several natural extensions of this framework merit investigation:

- **Quantizing the manifold (probabilistic runtimes):** Replacing the classical smooth manifold with a quantum mechanical state space (a Hilbert space or, more generally, a C^* -algebra) yields a probabilistic runtime. Measurement collapses the wave function — a nondeterministic operation — precisely at points of non-commutativity, mirroring the role of curvature in our framework.
- **Singular manifolds and exception handling:** Extending the framework to manifolds with singularities (stratified spaces, orbifolds, schemes over fields of characteristic p) provides a geometric model for exception handling and partial functions: singularities are the geometric locus of runtime errors, and blowing up a singularity corresponds to installing an exception handler.
- **Orbifolds and quotient type semantics:** An orbifold is a manifold with a discrete group action, yielding a quotient space with finite-order singularities at fixed points. This provides a geometric model for quotient types and setoid-based type theories: points in the quotient correspond to equivalence classes of program states, and the orbifold group encodes the equivalence relation.
- **Higher categorical structure:** The framework as presented is 1-categorical (morphisms are smooth maps between manifolds). Extending to $(\infty, 1)$ -categories via the framework of cohesive ∞ -topoi [Sch13] yields a homotopy-coherent runtime in which program equivalence is determined up to homotopy — the geometric analogue of observational equivalence in programming language theory.

References

6. [Cur86] P.-L. Curien. *Categorical Combinators, Sequential Algorithms, and Functional Programming*. Pitman, London, 1986.
7. [FGR16] L. Fajstrup, E. Goubault, and M. Raussen. *Directed Algebraic Topology: Models of Non-Reversible Worlds*. Cambridge University Press, Cambridge, 2016.
8. [Koc81] A. Kock. *Synthetic Differential Geometry*. London Mathematical Society Lecture Note Series, vol. 51. Cambridge University Press, Cambridge, 1981. Second edition, 2006.
9. [L03] J. M. Lee. *Introduction to Smooth Manifolds*. Graduate Texts in Mathematics, vol. 218. Springer-Verlag, New York, 2003. Second edition, 2013.
10. [Law79] F. W. Lawvere. "Categorical dynamics." In *Topos Theoretic Methods in Geometry*, A. Kock (Ed.), Aarhus University Various Publications Series, vol. 30, pp. 1–28, 1979.
11. [MS74] J. W. Milnor and J. D. Stasheff. *Characteristic Classes*. Annals of Mathematics Studies, vol. 76. Princeton University Press, Princeton, NJ, 1974.
12. [Sch13] U. Schreiber. "Differential cohomology in a cohesive ∞ -topos." Preprint, arXiv:1310.7930, 2013. Extended version, 2020.
13. [Shu18] M. Shulman. "Brouwer's fixed-point theorem in real-cohesive homotopy type theory." *Mathematical Structures in Computer Science*, 28(6):856–941, 2018.
14. [dR31] G. de Rham. "Sur l'analysis situs des variétés à n dimensions." *Journal de Mathématiques Pures et Appliquées*, 10:115–200, 1931.
15. [Bay26] A. Bayer. "Trajectory manifolds for nonlinear data-enabled predictive control: Existence, smoothness, and intrinsic dimension." Preprint, arXiv:2609.19079 [math.OC], 2026.
16. [GP74] V. Guillemin and A. Pollack. *Differential Topology*. Prentice-Hall, Englewood Cliffs, NJ, 1974.

- 17.[KN63] S. Kobayashi and K. Nomizu. *Foundations of Differential Geometry*, vols. I-II. Interscience Publishers, New York, 1963-1969.
- 18.[Lan96] R. Landauer. "The physical nature of information." *Physics Letters A*, 217(4-5):188-193, 1996. Originally presented 1961.
- 19.[Mac98] S. Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics, vol. 5. Springer-Verlag, New York, second edition, 1998.

Manifold Runtime Architecture · Technical Paper · 2026 | Author: Leon Calvi CC BY 4.0